

Homework 5: Semantics

Zimo Qi

Question 1

Question 2

Question 3

(a) (1). In lambda form: $\lambda x \text{ loves}(\text{Mary}, x)$ (2). Without any lambda: $\text{loves}(\text{Mary}, \cdot)$

(b) Because the VP “loves Mary” has such directed semantics?

$$\lambda x \text{ loves}(\text{Mary}, x)$$

(c) (1). In lambda form: $f = \lambda y (\forall x \text{ woman}(x) \Rightarrow \text{loves}(x, y))$ (2). In English: f : “is loved by every woman”; $f(\text{John})$: “John is loved by every woman.”

(d) $f = \lambda g \lambda x \text{ Obviously}(g(x))$

When constructing the semantics of the sentence “Sue obviously loves Mary”, we may first get the VP “loves Mary” with semantics $\lambda x. \text{loves}(\text{Mary}, x)$ in the deeper level. Then we want to add the adverb “obviously” corresponding to f , which modifies a VP as a new VP.

To realize it, we may “pop” the outer λx from the VP to get access to its inner meaning $\text{loves}(\text{Mary}, x)$, apply the modification (wrapping with $\text{Obviously}(\cdot)$), and then “push” λx back. Finally we obtain the new VP: $\lambda x. \text{Obviously}(\text{loves}(\text{Mary}, x))$, which can then apply to the subject “Sue”:

$$(\lambda x. \text{Obviously}(\text{loves}(\text{Mary}, x)))(\text{Sue}) = \text{Obviously}(\text{loves}(\text{Mary}, \text{Sue}))$$

(e) $f = \lambda y \lambda x (\lambda e \text{ act}(e, \text{loving}) \text{ lovee}(e, y) \text{ lover}(e, x))$

(f) $g = \lambda f \lambda y \lambda x (f(y)(x) \text{ manner}(x, \text{passionate}))$

The order of composition is ‘love Mary’ → ‘passionately’ → ‘John’. So first, in the deeper level, we have the meaning of “love Mary” as $\lambda x \lambda e \text{ act}(e, \text{loving}) \text{ lovee}(e, \text{Mary}) \text{ lover}(e, x)$. Then we try to apply g (the meaning of adverb “passionately”) to it.

To realize it, we *pop* the outer λx to access the event description, insert the new predicate $\text{manner}(e, \text{passionate})$, and then *push* λx back. Thus we get the result VP:

$$\lambda x \lambda e (\text{act}(e, \text{loving}) \text{ lovee}(e, \text{Mary}) \text{ lover}(e, x) \text{ manner}(e, \text{passionate}))$$

(g) (1): $f = \lambda g (\forall y \text{ woman}(y) \Rightarrow g(y))$ (2): Translations: i: $f(\lambda x. \text{loves}(\text{Mary}, x)) =$ “Every woman loves Mary” ii: $(\lambda x \text{ loves}(\text{Mary}, x)) =$ “loves Mary” iii: $f =$ “every woman”,

(h) (1): $g = \lambda Q \lambda P (\forall y Q(y) \Rightarrow P(y))$ (2): It represents the English word “every”.

(i) (1): $f = \lambda g g(\text{Papa})$ (2): This function f takes a VP meaning g and applies it to the constant Papa , yielding $P(\text{Papa}) = \text{loves}(\text{Mary}, \text{Papa})$.

We assign $(NP \text{ Papa})$ the semantics $\lambda P P(\text{Papa})$ rather than simply the simple meaning Papa to keep that proper names consistent to other semantic type as quantified NPs, like “every woman”: $\lambda P \forall y \text{ woman}(y) \Rightarrow P(y)$. This allows the grammar to handle all NPs compositionally in the same way.

Question 4

Appropriate Parses Most sentences received appropriate semantic interpretations, with the expected tense, argument order, and clause-type operators.

- **Joe love -s Jill .**
`assert(pres(love(Jill,Joe)), speaker) — CORRECT.`
- **he love -s her .**
`assert(pres(love(her,him)), speaker) — CORRECT.`
- **Papa sleep -s with a spoon .**
`assert(pres(with(some(spoon),sleep(Papa))), speaker) — CORRECT.`
- **Papa eat -ed with a spoon .**
`assert(past(with(some(spoon),eat(something,Papa))), speaker) — CORRECT.`
- **have a bonbon !**
`command(possess(some(bonbon))(hearer), speaker) — CORRECT.`
- **a bonbon on the spoon entice -s .**
`assert(pres(entice(something,some(%x bonbon(x) ^ on(the(spoon),x)))), speaker) — CORRECT.`
- **the bonbon -s on the spoon entice -0 .**
`assert(pres(entice(something,the_plural(%x bonbon(x) ^ on(the(spoon),x)))), speaker) — CORRECT.`
- **Joe kiss -ed every chief of staff .**
`assert(past(kiss(all(chief_of_staff),Joe)), speaker) — CORRECT.`
- **the perplexed president eat -ed a pickle .**
`assert(past(eat(some(pickle),the(%x president(x) ^ perplexed(x)))), speaker) — CORRECT.`
- **Papa is perplexed .**
`assert(pres(perplexed(Papa)), speaker) — CORRECT.`
- **Papa is chief of staff .**
`assert(pres(chief_of_staff(Papa)), speaker) — CORRECT.`
- **Papa want -ed a sandwich .**
`assert(past(want(some(sandwich),Papa)), speaker) — CORRECT.`
- **Papa want -ed to eat a sandwich .**
`assert(past(want(eat(some(sandwich),Papa), Papa)), speaker) — CORRECT.`
- **Papa want -ed Joe to eat a pickle .**
`assert(past(want(eat(some(pickle),Joe), Papa)), speaker) — CORRECT.`

- **Papa want -ed a pickle to eat Joe .**
`assert(past(want(eat(Joe,some(pickle)), Papa)), speaker) — grammatically valid but semantically odd; the system still provides a structurally correct interpretation.`
- **Papa would have eat -ed his sandwich -s .**
`assert(would(perf(eat(the_plural(%x sandwich(x)^possess(x,him)),Papa))), speaker) — CORRECT.`
- **every sandwich was go -ing to have been delicious .**
`assert(past(prog(will(perf(delicious(all(sandwich)))))), speaker) — CORRECT.`

Inappropriate or Failed Parses A few sentences produced either incorrect semantic attachment or no consistent attribute assignment:

- **he love -s she .**
`assert(pres(love(her,him)), speaker) — INAPPROPRIATE.` Grammatically incorrect (“she” should be in objective case). However, the parser still returns a plausible meaning. It lacks such rules to handle this.
- **Papa sleep -s every bonbon with a spoon .**
FAILED: No consistent way to assign attributes. It correctly rejected the sample sentence that attaching NP to intransitive verb *sleep*.
- **Papa eat -ed every bonbon with a spoon .**
`assert(past(eat(all(%x bonbon(x) ^ with(some(spoon),x)),Papa)), speaker) — INAPPROPRIATE.` The PP “with a spoon” modifies the noun “bonbon” instead of the verb “eat”.
- **Jill say -s that Joe might sleep on the floor !**
`exclaim(pres(on(the(floor),say(might(sleep(Joe)),Jill))), speaker) — INAPPROPRIATE.` The PP “on the floor” attaches to the clause “say,” rather than to the embedded verb “sleep.”
- **a bonbon on the spoon entice -0 .**
FAILED: No consistent way to assign attributes. The system fails due to singularity and plurality mismatch, as expected.
- **the fine and blue woman and every man must have eat -ed two sandwich -s and sleep -ed on the floor .**
`assert(must(on(the(floor),perf(eat(two(sandwich),and(the(%x woman(x) ^ (fine(x)^blue(x))),all(man))))),sleep(and(the(%x woman(x) ^ (fine(x)^blue(x))),all(man))))), speaker) — INAPPROPRIATE.` The PP “on the floor” scopes over the entire conjunction, giving the reading “they ate and slept on the floor,” whereas only “sleep” should be modified by this PP.

Question 5

Modifications. I copied the original grammar into `english2.gra` and added the following changes:

- New noun:

```
1 N[=1 num=mass sem=caviar] caviar
```
- Extended determiners for mass nouns:

```
1 Det[=1 num=mass sem=all_mass] all
1 Det[=1 num=mass sem=some_mass] some
1 Det[=1 num=mass sem=the_mass] the
```

- Verb treating mass nouns as singular to build a S:

```
1 S[=2 sem=pres(2(1))] NP[num=mass] VP[num=sing tense=pres]
1 S[=2 sem=past(2(1))] NP[num=mass] VP[num=sing tense=past]
1 S[=2 sem=2(1)] NP[num=mass] VP[num=sing tense=modal]
```

- Mass N promotion to NP, no need Det:

```
1 NP[=1 num=mass sem=some(1)] N[num=mass]
```

Testing

- *Papa eat -s caviar.* → grammatical; `sem=assert(pres(eat(caviar,Papa)),speaker).`
- *all caviar is delicious.* → grammatical; `sem=assert(pres(delicious(all_mass(caviar))), speaker).`
- *some caviar is delicious .* → grammatical; `sem=assert(pres(delicious(some_mass(caviar))), speaker)`
- *The caviar is delicious.* → grammatical; `sem=assert(pres(delicious(the_mass(caviar))), speaker)`
- *the caviar are delicious.* → ungrammatical (No consistent way to assign attributes!).
- *Caviar are delicious.* → ungrammatical (No consistent way to assign attributes!).

Discussion. (1)Mass nouns are treated syntactically like singular nouns in verb agreement, (2)Mass nouns share determiner compatibility with plural nouns.

One possible obstacle is that the non-attributed parser sometime gives wrong results-skipping the necessary procedure NP<-N and directly use the `1 VP[=1 sem=1(2)] V[arg=pred] N` which is originally designed expression like "be president". To handle this, I have to increase the weight for `VP[=1 sem="1(2)"] V[arg=np] NP` that force VP to be created by V and NP.

Question 6

(a) Interpreting the determiner semantics

- **two:**

```
Det[... , sem="%dom %pred E%first E%second [first!=second ^ dom(first)^dom(second)]
^ pred(first) ^ pred(second)"] two
```

- λ **explanation:** $\lambda p \lambda q \exists x \exists y [x \neq y \wedge p(x) \wedge p(y) \wedge q(x) \wedge q(y)]$.

- **Natural Language Explanation:** There are two functions that there exist two individuals who share the same value on the two functions / There exist two individuals who belong to the same domain and share the same particular predicate.

- **the (singular):**

```
Det[num=sing, sem="%dom %pred E%t [dom(t) ^ A%u [dom(u) ^ u!=t ==> salience(u) < salience(t)]
^ pred(t)"] the
```

- $\lambda p \lambda q \exists t [p(t) \wedge \forall u (p(u) \wedge u \neq t \Rightarrow \text{salience}(u) < \text{salience}(t)) \wedge q(t)]$

- **Natural Language Explanation:** There are two functions (1 and 2) that there exist particular one individual who is more significant (to some extent) than others with the same function 1 value, and the individual has function 2 value / There exists one particular individuals who is more significant (to some extent) than others in the same domain and has an predicate.

- **the (plural):**

Det[num=pl sem="%dom, %pred E%T [subset(T,dom) $\wedge$ |T|>one $\wedge$ A%U [subset(U,dom) $\wedge$ |U|>one $\wedge$ U!=T ==> salience(U) < salience(T)]] $\wedge$ pred(T)"] the

- λ **explanation:** $\lambda p \lambda q \exists T [subset(T,p) \wedge |T| > 1 \wedge \forall U (subset(U,p) \wedge |U| > 1 \wedge U \neq T \Rightarrow salience(U) < salience(T)) \wedge q(T)]$

- **Natural Language Explanation:** There are two functions (1 and 2) that there exist one particular set of individuals who is more significant (to some extent) than any other sets with the same function 1 value, and the set of individuals has function 2 value / There exists one particular set of individuals who is more significant (to some extent) than any other sets of individuals in the same domain and they have the same predicates.

(b) I set the missing semantics to:

"%subj 2(%obj 1(obj,3,subj))",

Explanation. At this step, we need to combine the semantics of a **V**, a **NP**, and a **VP** into a new VP, according to the rule

VP[=1 tense=1 sem="%subj 2(%obj 1(obj,3,subj))"] V[arg=npvpinf] NP VP[tense=inf].

By tracing the parsing process, we have already obtained the following constituent semantics:

1. **V:** *want -ed*

%obj %pred %subj want(pred(obj), subj)

2. **NP:** *Joe*

%pred pred(Joe)

3. **VP:** *to eat a pickle*

%subj E%b [pickle(b)] $\wedge$ eat(b,subj)

The goal is for the new VP (the mother node) to have only one free variable %subj, representing the subject NP(*Papa*). **First**, considering the constituent *want -ed*, the first and third variables are objects while the second variable is a function. So we should substitute 3 directly into the second variable, say 1(xxx, 3, xxx). **Second**, we parse the whole 1 to 2, get the correct predicate. Also, we leave the third variable of 1 for following subjective. So the semantics will be in the form of %subj 2(1(xxx, 3, subj)). **Third**, we need to write parameter into 2 as a function form and also need to explicitly give the subj of 3, that is the objective in 1. Thus we have %subj 2(%obj 1(obj, 3, subj))

After substitution, the resulting semantics becomes:

%subj want(E%b [pickle(b)] $\wedge$ eat(b,Joe), subj)

and the final result is:

assert(past(want(E%b [pickle(b)] $\wedge$ eat(b, Joe), Papa)), speaker)

which I think is appropriate. I have to say this question is quite tricky, lol.